

**NATIONAL UNIVERSITY OF MODERN
LANGUAGE ISLAMABAD**

Assignment : 02



Submitted to:

Mr. Danish

Submitted by:

Zulqarnain Hassan

Roll No: SP21379

Subject: WEB II

Department: BSSE-7-Afternoon

Date: 23-12-2026

INTERFACE OF PROJECT:

The user interface (UI) for the BreatheWell system was developed with a strong focus on calmness, clarity, and trust, which are paramount for a health monitoring application. The design philosophy centers on making complex AI analysis feel approachable and easy for the user.

Design Theme and Aesthetic

The application employs a clean, modern, and health-centric aesthetic.

- **Color Palette:** Dominated by a crisp white background for optimal readability, complemented by soothing teal and turquoise accents (as seen in the logo and feature icons). This palette evokes feelings of tranquility, freshness, and clinical trust.
- **Typography:** A legible, clean sans-serif font is used to ensure all medical and tracking information is easily digestible.
- **Imagery:** The use of clear, modern illustrations (like the one showing lung health) immediately communicates the app's function while maintaining a friendly, non-intimidating tone.

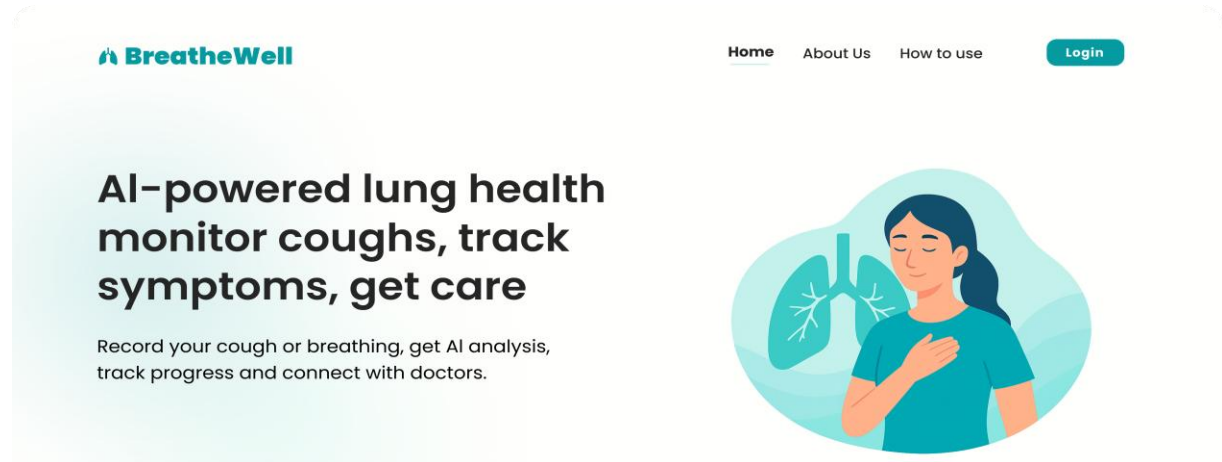
Key UX/UI Decisions.

The visual elements are structured for immediate comprehension:

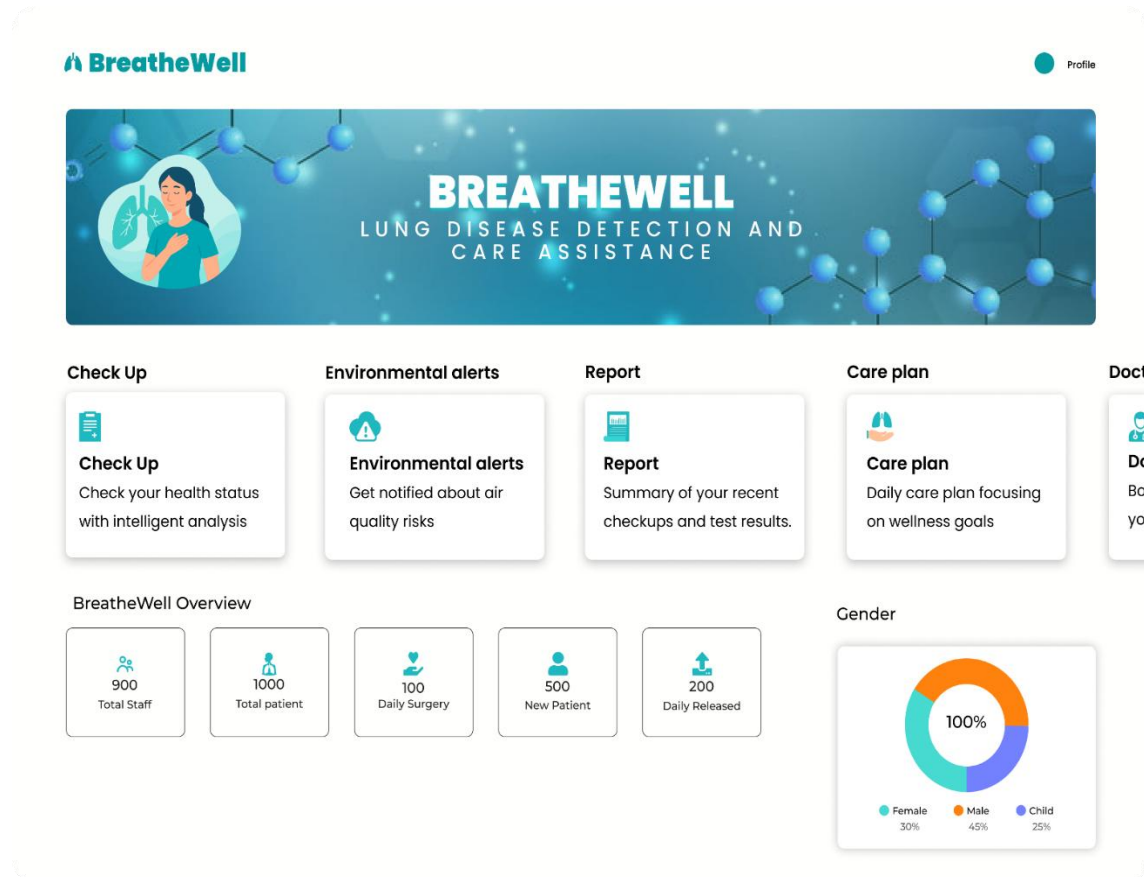
1. **Clear Value Proposition:** The headline, "AI-powered lung health monitor coughs, track symptoms, get care," is prominently placed (as highlighted in your design) to instantly inform the user of the app's core services.
2. **Feature Accessibility:** The main features (Audio analysis, Symptom tracker, Environmental alerts, Doctor connect) are presented as distinct, easy-to-read cards. This card-based layout makes complex services scannable and accessible.
3. **Minimalist Navigation:** The top navigation (Home, About Us, How to use, Login) is kept minimal to reduce visual clutter and focus the user on the primary monitoring functions.

The following screens provide a detailed look at the interface, demonstrating how these design principles translate into a functional and effective user experience.

Landing page



Dashboard page.



Codes:

App.py:

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from pymongo import MongoClient
from werkzeug.security import generate_password_hash, check_password_hash
app = Flask(__name__)
CORS(app)
client = MongoClient("mongodb://localhost:27017/") # Replace with your MongoDB
URI if using Atlas
db = client.breathewell_db

doctors_collection = db.doctors
patients_collection = db.patients
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    required_fields = ["fullName", "email", "password", "confirmPassword", "age",
"gender", "height", "weight", "role"]
    is_invalid = False
    for field in required_fields:
        if field not in data or data[field] == "":
            is_invalid = True
            break
    if is_invalid:
        return jsonify({"error": "please fill all fields"}), 400
    if data["role"] not in ["doctor", "patient"]:
        return jsonify({"error": "Role must be doctor or patient"}), 400
    if data["password"] != data["confirmPassword"]:
        return jsonify({"error": "Passwords do not match"}), 400
    collection = doctors_collection if data["role"] == "doctor" else
patients_collection
    if collection.find_one({"email": data["email"]}):
        return jsonify({"error": f"{data['role'].capitalize()} already exists"}),
400
    hashed_password = generate_password_hash(data["password"])
    user_data = {
        "fullName": data["fullName"],
        "email": data["email"],
        "password": hashed_password,
        "age": data["age"],
        "gender": data["gender"],
        "height": data["height"],
```

```

        "weight": data["weight"],
    }

    collection.insert_one(user_data)

    return jsonify({"message": f"{data['role'].capitalize()} registered successfully"}), 201

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    email = data.get("email")
    password = data.get("password")
    role = data.get("role") # Role must be sent from frontend

    if not email or not password or not role:
        return jsonify({"error": "Please provide email, password, and role"}),
400

    if role not in ["doctor", "patient"]:
        return jsonify({"error": "Role must be doctor or patient"}), 400
    collection = doctors_collection if role == "doctor" else patients_collection
    user = collection.find_one({"email": email})

    if not user:
        return jsonify({"error": f"{role.capitalize()} not found"}), 404

    if not check_password_hash(user["password"], password):
        return jsonify({"error": "Incorrect password"}), 401
    user_data = {
        "fullName": user["fullName"],
        "email": user["email"],
        "age": user["age"],
        "gender": user["gender"],
        "height": user["height"],
        "weight": user["weight"],
        "role": role
    }
    return jsonify({"message": "Login successful", "user": user_data}), 200
if __name__ == '__main__':
    app.run(debug=True, port=5000)

```